Lab05-TM&Complexity

CS2312-Algorithm and complexity, Xiaofeng Gao, Spring 2026.

∗ Contact TA Steven Chen for any questions. Include both your .pdf and .tex files in the uploaded .rar or .zip file.

∗ Name:_____ Student ID:_____ Email: _____

1. (Turing Machine)

Given two sorted non-empty 3-bit binary integer arrays $nums_1$ and $nums_2$ in non-decreasing order, construct a 3-tape Deterministic Turing Machine that outputs the merged sorted array in non-decreasing order. The input tape is read-only.

Initial Configuration.

$q_s \rightarrow$	$\triangleright$	0	0	1	|	0	1	0	|	0	1	1	#	0	1	0	|	1	0	1	|	1	1	0	$\triangleleft$
$q_s \rightarrow$	$\triangleright$																							
$q_s \rightarrow$	$\triangleright$																							

Alphabets. $\Gamma = \{0, 1, \triangleright, \ \triangleleft, \ |, \ \#, \square\}$

- $\triangleright$ denotes the left-end marker,
- $\triangleleft$ denotes the right-end marker,
- # denotes the delimiter between arrays,
- | denotes the delimiter between integers in an array,
- $\square$ denotes blank.

(a) Design an instruction set of the form:

$$\langle q_i, s^1_{j_1}, s^2_{j_2}, s^3_{j_3} \rangle \rightarrow \langle q_l, s^2_{k_2}, s^3_{k_3}, L/R/S, L/R/S, L/R/S \rangle.$$

For example, suppose the machine is currently in state q_s, and all three tape heads are scanning the left-end marker $\triangleright$. We may move all three tape heads one cell to the right without changing the tape contents, while transitioning to state q_1. The corresponding instruction is:

$$\langle q_s, \triangleright, \triangleright, \triangleright \rangle \rightarrow \langle q_1, \triangleright, \triangleright, R, R, R \rangle$$

(b) Briefly show the whole process from the initial to the final configuration, where the input arrays are $nums_1 = [1, 2, 3]$ and $nums_2 = [2, 5, 6]$, and the output array is $[1, 2, 2, 3, 5, 6]$. For example:

$$(q_s, \trianglerighteq 001|010|011\#010|101|110\triangleleft, \trianglerighteq, \trianglerighteq) \vdash (q_1, \triangleright \underline{0}01|010|011\#010|101|110\triangleleft, \trianglerighteq, \trianglerighteq)$$

$$\vdash \cdots \vdash (q_h, \triangleright 001|010|011\#010|101|110\underline{\triangleleft}, \cdots, \triangleright 001|010|010|011|101|110\underline{\triangleleft})$$

Note: You do not have to write out the non-transitioning tapes fully. For example, if only tape 1 changes state, you may write

$$(q_s, \trianglerighteq 001|010|011\#010|101|110\triangleleft, \trianglerighteq, \trianglerighteq) \vdash (q_1, \triangleright \underline{0}01|010|011\#010|101|110\triangleleft, \ldots, \ldots)$$

If the same instruction executes repeatedly, you may skip the intermediate transitions. For example, if your instruction consecutively shifts the first tape pointer to the right, you may skip from

$$(q_s, \trianglerighteq 001|010|011\#010|101|110\triangleleft, \trianglerighteq, \trianglerighteq) \vdash^* (q_1, \triangleright 001|010|011\underline{\#}010|101|110\triangleleft, \trianglerighteq, \trianglerighteq)$$

2. (Reduction via Gadgets)

The Steiner Tree (ST) problem: Given an undirected, unweighted, connected graph $G = (V, E)$, and a subset S of the nodes of G (the shaded nodes), plus a non-negative integer k, is there a subset E' of edges of G that connects all of the shaded nodes, where $|E'| \leq k$?

(a) Show that the ST problem is in NP.

(b) Describe an efficient reduction from the 3-SAT to the ST problem.

(c) Prove the correctness of your reduction by showing that instance Φ is a Yes-instance of 3-SAT if and only if instance $I = (G, S, k)$ is a Yes-instance of ST.

3. (NP-Complete)

A recruitment agency wishes to offer n possible jobs. There are k job seekers interested in getting a job and each is interested in some subset of the n possible jobs. To manage costs, the agency has a bound b on the number of jobs it is willing to offer. Is there a way to choose b jobs out of the n possibilities so that every job seeker can get a job that interests them? Formally, an instance of the Job Scheduling (JS) problem consists of

- a set C of n possible courses, numbered 1 through n,
- subsets $S_1, S_2, \ldots, S_k$ of $\{1, 2, \ldots, n\}$ (the jobs of interest to each of the job seekers),
- and a bound b (the maximum number of jobs actually offered).

The JS problem is to determine whether there is a subset B of $\{1, 2, \ldots, n\}$, where the size of $B \leq b$, such that $B \cap S_i$ is not empty for all $i, 1 \leq i \leq k$.

(a) Describe an efficient reduction from the Vertex Cover problem to the JS problem.

(b) Show that your reduction runs in polynomial time.

(c) Show that the JS problem is NP-complete.